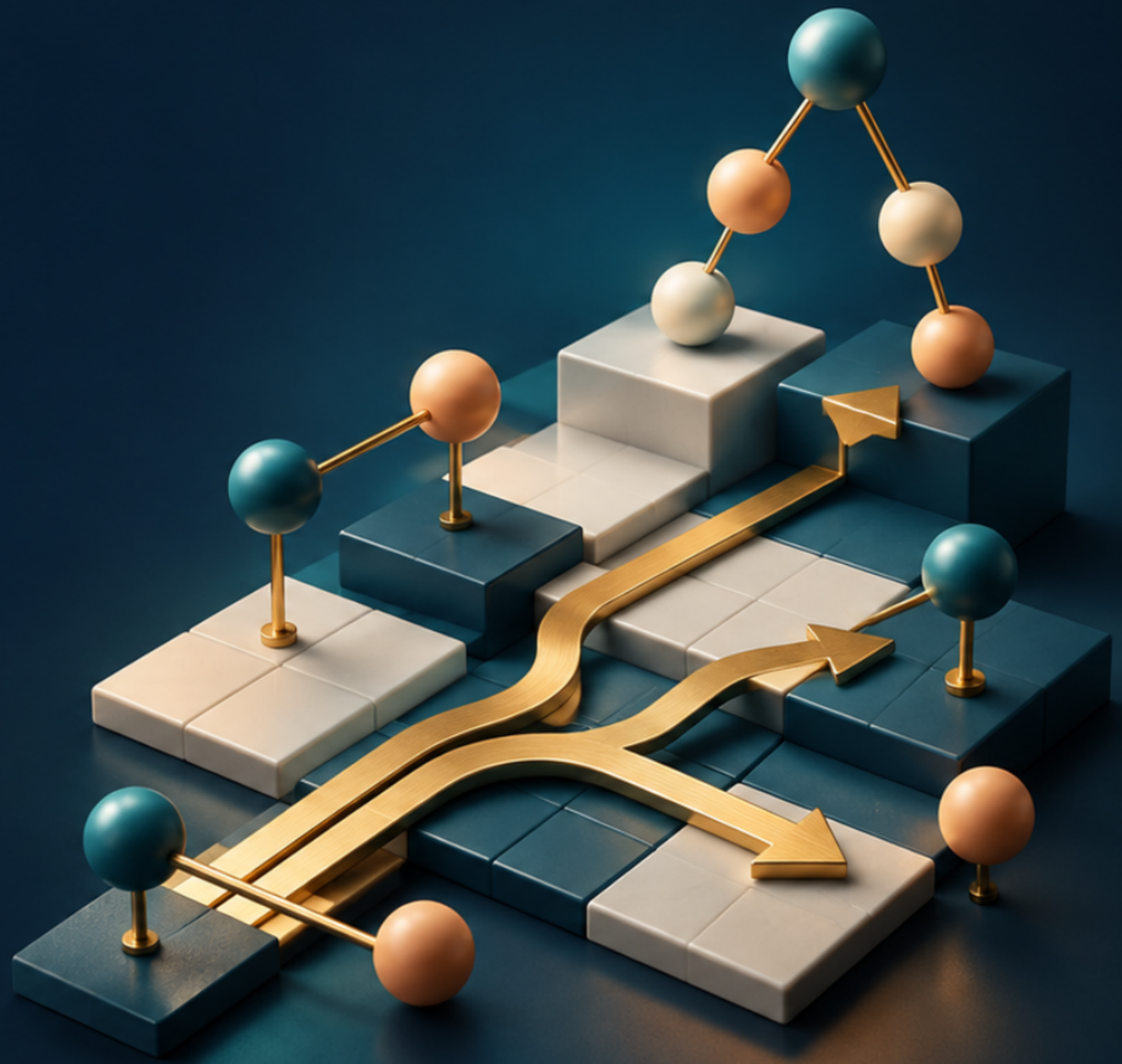
06

동적 계획법

계단과 숫자 구간을
한 단계씩 계산합니다.



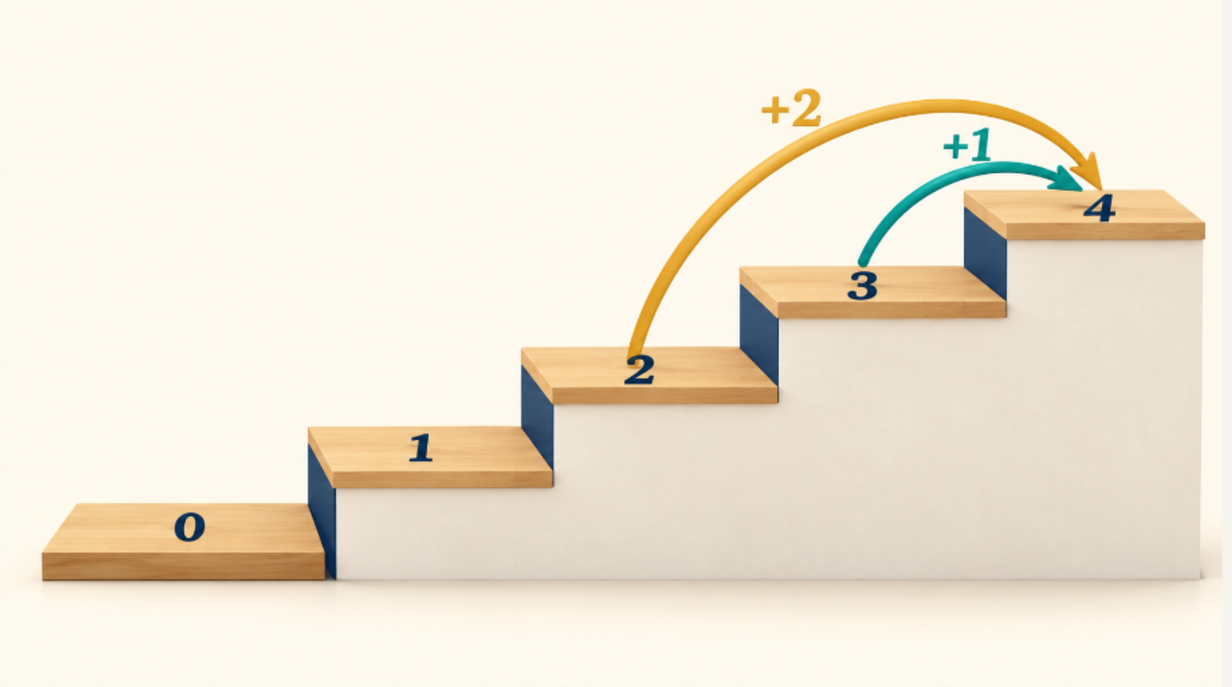
오늘 풀 두 문제

먼저 손으로 풀고, 그 계산을 그대로 코드로 옮깁니다.

문제	작은 입력	구할 답
6-2 계단 오르기	높이 4칸	오르는 순서 5가지
6-3 연속 부분합	4, −6, 3, 2, −1, 4	가장 큰 구간 합 8

6-2 계단 오르기

한 번에 1칸 또는 2칸. 순서가 다르면 다른 방법입니다.



입력: 계단 높이 n　　**구할 것: 꼭대기까지 오르는 방법 수**

2칸 계단은 두 가지

오르는 순서	도착 높이
1 + 1	2칸
2	2칸

2가지 서로 다른 순서가 있으므로 답은 2입니다.

3칸 계단은 세 가지

오르는 순서	도착 높이
1 + 1 + 1	3칸
1 + 2	3칸
2 + 1	3칸

3가지 서로 다른 순서가 있으므로 답은 3입니다.

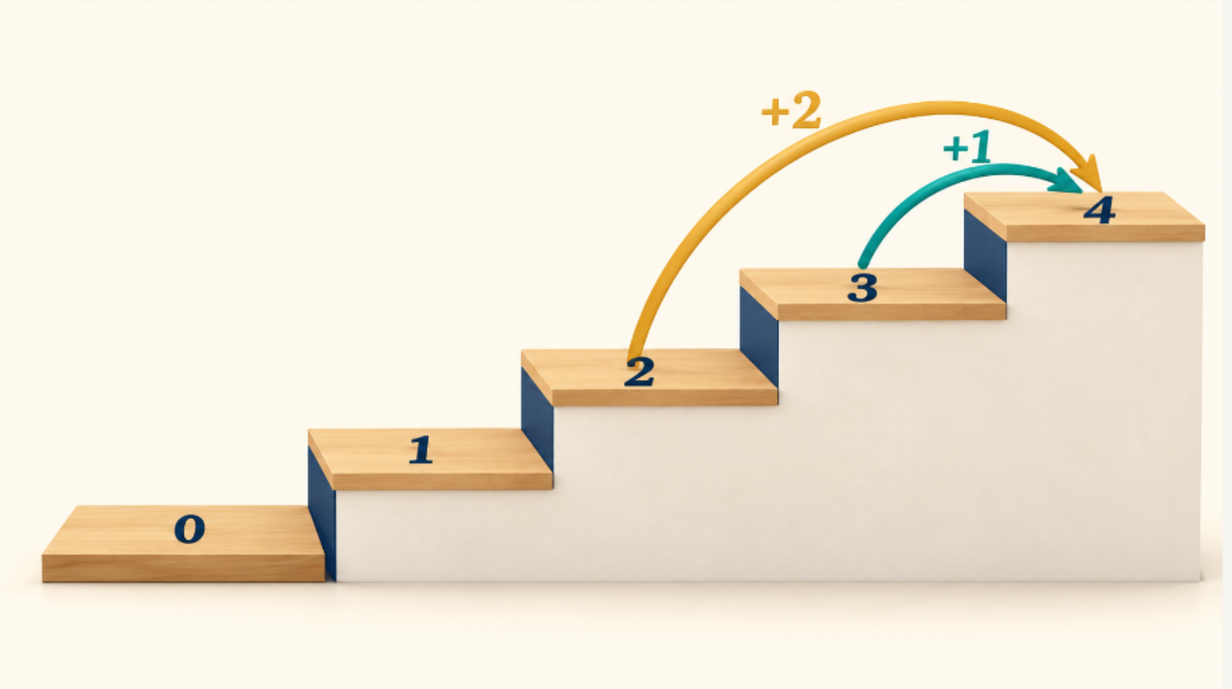
4칸 계단은 다섯 가지

오르는 순서	도착 높이
1 + 1 + 1 + 1	4칸
1 + 2 + 1	4칸
2 + 1 + 1	4칸
1 + 1 + 2	4칸
2 + 2	4칸

5가지 서로 다른 순서가 있으므로 답은 5입니다.

마지막 이동은 두 경우뿐

3칸에서 1칸 더 오르거나, 2칸에서 2칸 더 오릅니다.



초록: 마지막 1칸　　**주황: 마지막 2칸**

3칸의 세 방법에 마지막 1칸을 붙이면

3칸까지의 순서	마지막 이동	4칸에 도착한 순서
1, 1, 1	1칸	1, 1, 1, 1
1, 2	1칸	1, 2, 1
2, 1	1칸	2, 1, 1

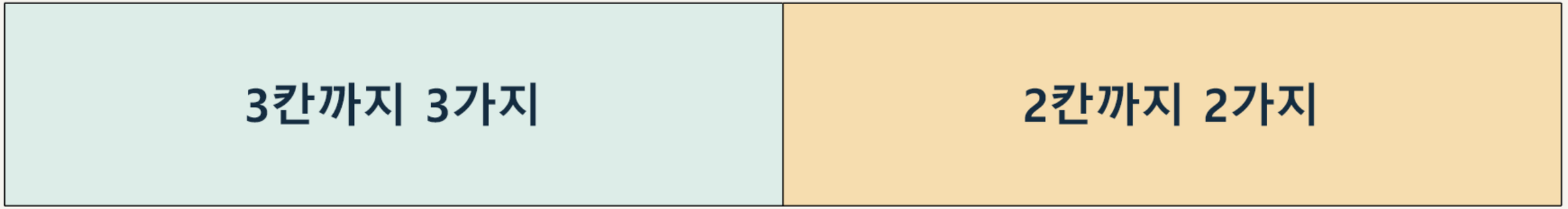
기존 3가지에 하나씩 붙이므로 그대로 3가지

2칸의 두 방법에 마지막 2칸을 붙이면

2칸까지의 순서	마지막 이동	4칸에 도착한 순서
1, 1	2칸	1, 1, 2
2	2칸	2, 2

기존 2가지에 하나씩 붙이므로 그대로 2가지

그래서 4칸의 답은 3 + 2



$$f(4) = f(3) + f(2) = 3 + 2 = 5$$

f(k): k칸까지 오르는 방법 수

5칸도 같은 방식으로 셉니다

구할 높이	마지막 1칸을 붙임	마지막 2칸을 붙임	방법 수
4칸	3칸의 3가지	2칸의 2가지	3 + 2 = 5
5칸	4칸의 5가지	3칸의 3가지	5 + 3 = 8
6칸	5칸의 8가지	4칸의 5가지	8 + 5 = 13

n칸의 방법 수 = (n−1)칸의 방법 수 + (n−2)칸의 방법 수

0칸의 답을 1로 두는 이유

높이	인정하는 순서	방법 수
0칸	아무 이동도 하지 않음	1
1칸	1칸 이동	1
2칸	1 + 1 또는 2	1 + 1 = 2

시작값: f(0) = 1, f(1) = 1

답을 적어 둘 칸에 dp라는 이름을 붙입니다

각 칸에는 그 높이까지 오르는 방법 수를 저장합니다.

높이 k	0	1	2	3	4
방법 수	1	1	2	3	**5**

dp[4] = 5 : 4칸까지 오는 방법은 5가지

n = 4이면 답을 담을 칸은 5개

각 칸에는 그 높이까지 오르는 방법 수를 저장합니다.

높이 k	0	1	2	3	4
방법 수	1	1	?	?	?

먼저 dp[0] = 1, dp[1] = 1을 적습니다.

2칸의 답을 채우는 차례

각 칸에는 그 높이까지 오르는 방법 수를 저장합니다.

높이 k	0	1	2	3	4
방법 수	1	1	2	?	?

초록: 읽는 두 답　　주황: 이번에 채우는 답

dp[2] = dp[1] + dp[0] = 1 + 1 = 2

3칸의 답을 채우는 차례

각 칸에는 그 높이까지 오르는 방법 수를 저장합니다.

높이 k	0	1	2	3	4
방법 수	1	1	2	3	?

초록: 읽는 두 답　　주황: 이번에 채우는 답

dp[3] = dp[2] + dp[1] = 2 + 1 = 3

4칸의 답을 채우는 차례

각 칸에는 그 높이까지 오르는 방법 수를 저장합니다.

높이 k	0	1	2	3	4
방법 수	1	1	**2**	**3**	**5**

초록: 읽는 두 답　　주황: 이번에 채우는 답

dp[4] = dp[3] + dp[2] = 3 + 2 = 5

지금 한 순서가 바텀업입니다

먼저 아는 답	그다음 구하는 답	마지막으로 읽는 답
높이 0과 1	높이 2, 3, 4	높이 4의 답 5

Bottom-up: 작은 높이의 답부터 차례대로 계산

계단 오르기 Python 전체 코드

```python
def climb_stairs(n):
    dp = [0] * (n+1)
    dp[0] = 1
    if n >= 1:
        dp[1] = 1
    for i in range(2, n+1):
        dp[i] = dp[i-1] + dp[i-2]
    return dp[n]
```

사이트 제출 방식: 바텀업. 함수 안에 dp 배열을 만들고 반복문으로 채웁니다.

i가 바뀔 때 읽는 칸도 바뀝니다

i	오른쪽에서 읽음	더한 값	왼쪽에 기록
2	dp[1]=1, dp[0]=1	2	dp[2]=2
3	dp[2]=2, dp[1]=1	3	dp[3]=3
4	dp[3]=3, dp[2]=2	5	dp[4]=5

for i in range(2, n+1) → i = 2, 3, …, n

C에서는 왜 long long을 쓸까요?

값의 종류	예시	C 자료형
계단 높이	n = 46	int
46칸의 방법 수	2,971,215,073	long long
32비트 int의 최댓값	2,147,483,647	방법 수가 범위를 넘음

큰 정수를 저장할 공간이 필요해서 long long을 사용합니다.

계단 오르기 C 전체 코드

```c
long long climb_stairs(int n) {
    long long dp[91] = {0};
    dp[0] = 1;
    if (n >= 1) dp[1] = 1;
    for (int i=2; i<=n; ++i) {
        dp[i] = dp[i-1] + dp[i-2];
    }
    return dp[n];
}
```

사이트 제출 방식: 바텀업. Python과 같은 표를 반복문으로 채웁니다.

C 반복문을 세 부분으로 읽으면

C 표현	읽는 말	n=4에서의 움직임
int i=2	2번 칸부터 시작	i는 처음에 2
i<=n	n번 칸까지 계산	2, 3, 4에서 실행
++i	다음 칸으로 이동	2 다음 3, 3 다음 4

for (int i=2; i<=n; ++i)

이번에는 4칸의 답부터 물어봅시다

지금 받은 질문	대신 필요한 작은 질문
4칸의 방법은?	3칸의 방법 + 2칸의 방법
3칸의 방법은?	2칸의 방법 + 1칸의 방법
2칸의 방법은?	1칸의 방법 + 0칸의 방법

큰 질문을 더 작은 질문으로 바꿔 물어봅니다.

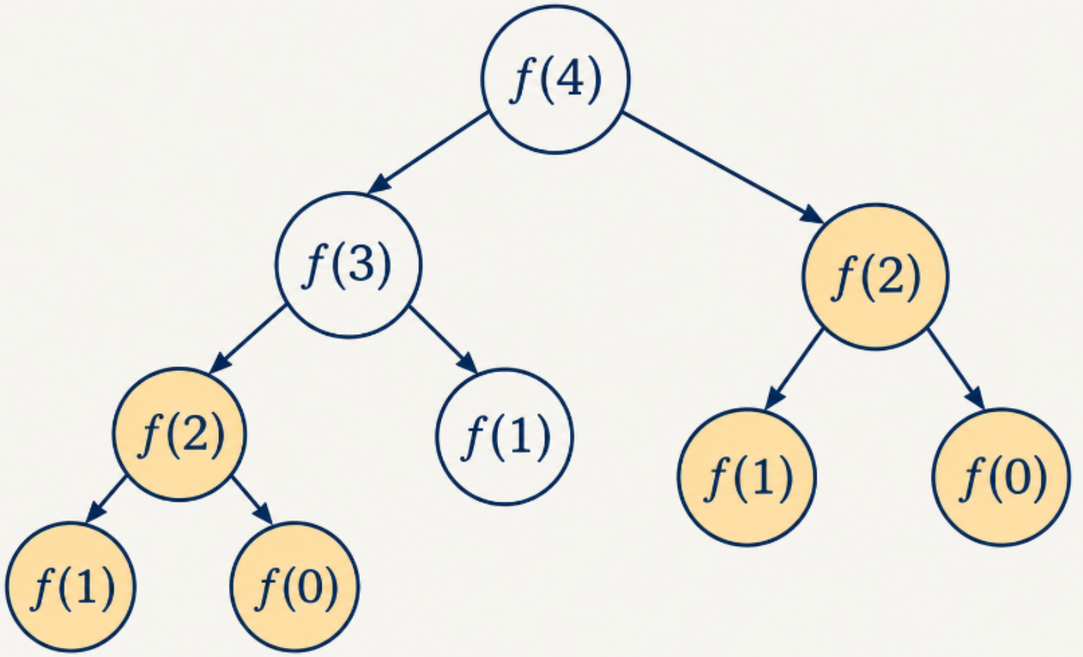
식을 그대로 재귀 함수로 쓰면

```
def f(k):
    if k <= 1:
        return 1
    return f(k-1) + f(k-2)
```

정답은 나오지만, 같은 계산을 반복합니다.

f(2)를 두 번 계산하고 있습니다

주황색 두 묶음은 질문도 답도 같습니다.



처음 구한 f(2) = 2를 저장하면, 다음에는 바로 읽을 수 있습니다.

답을 구했으면 메모해 둡니다

요청	메모 확인	실제로 하는 일
처음 f(2)	아직 답이 없음	1 + 1을 계산하고 2를 저장
다시 f(2)	높이 2의 답은 2	저장한 2를 바로 반환

메모이제이션: 같은 입력의 계산 결과를 저장하고 재사용

Python 사전은 이름표와 답을 함께 저장합니다

표현	하는 일	결과
memo = {0: 1, 1: 1}	시작 답 두 개를 적어 둠	0칸은 1, 1칸도 1
memo[2] = 2	높이 2의 답을 저장	2칸은 2
2 in memo	높이 2가 저장됐는지 확인	True
memo[2]	높이 2의 답을 읽음	2

키 = 계단 높이　　**값 = 그 높이까지 오르는 방법 수**

비교용: memo는 바깥 함수에서 한 번 만듭니다

```
def climb_stairs(n):
    memo = {0: 1, 1: 1}

    def f(k):
        # 같은 memo를 읽고 채움
        ...

    return f(n)
```

memo는 한 번 생성

f 호출끼리 공유

재귀 호출은 f로

탑다운의 구조를 보는 예시입니다. 사이트에서는 바텀업 반복문을 사용합니다.

비교용: 메모를 쓰는 탑다운 코드

```
def climb_stairs(n):
    memo = {0: 1, 1: 1}

    def f(k):
        if k in memo:
            return memo[k]
        memo[k] = f(k-1) + f(k-2)
        return memo[k]

    return f(n)
```

탑다운 비교 예시입니다. 사이트의 계단 문제는 바텀업으로 작성합니다.

질문은 4, 3, 2의 순서로 내려갑니다

질문한 함수	필요한 답	지금 상태
f(4)	f(3), f(2)	f(3)의 답을 기다림
f(3)	f(2), f(1)	f(2)의 답을 기다림
f(2)	f(1), f(0)	두 답 모두 메모에 1

먼저 작은 질문까지 내려간 뒤, 답을 더하며 돌아옵니다.

답은 2, 3, 4칸 순서로 완성됩니다

차례	계산 또는 읽기	메모에 남는 답
1	f(2) = 1 + 1	2칸: 2
2	f(3) = 2 + 1	3칸: 3
3	**f(2)의 저장값을 읽음**	**2를 그대로 사용**
4	f(4) = 3 + 2	4칸: 5

같은 f(2)가 다시 나와도, 저장한 답을 읽습니다.

두 방식은 시작하는 방향이 다릅니다

비교	탑다운 Top-down	바텀업 Bottom-up
시작	구하려는 4칸부터 질문	이미 아는 0칸, 1칸
진행	필요한 작은 답을 요청	2칸, 3칸, 4칸을 계산
저장	함수 결과를 메모	표를 순서대로 채움
구현	재귀 호출	반복문

둘 다 f(4) = 5. 필요한 답을 구하고 저장하는 순서가 다릅니다.

한 번 구한 답을 쓰면 반복 계산이 줄어듭니다

높이 n	메모 없는 재귀: 덧셈 횟수	메모 사용: 덧셈 횟수
4	4	3
10	88	9
20	10,945	19

탑다운 + 메모, 바텀업 표 채우기: 모두 시간 O(n)

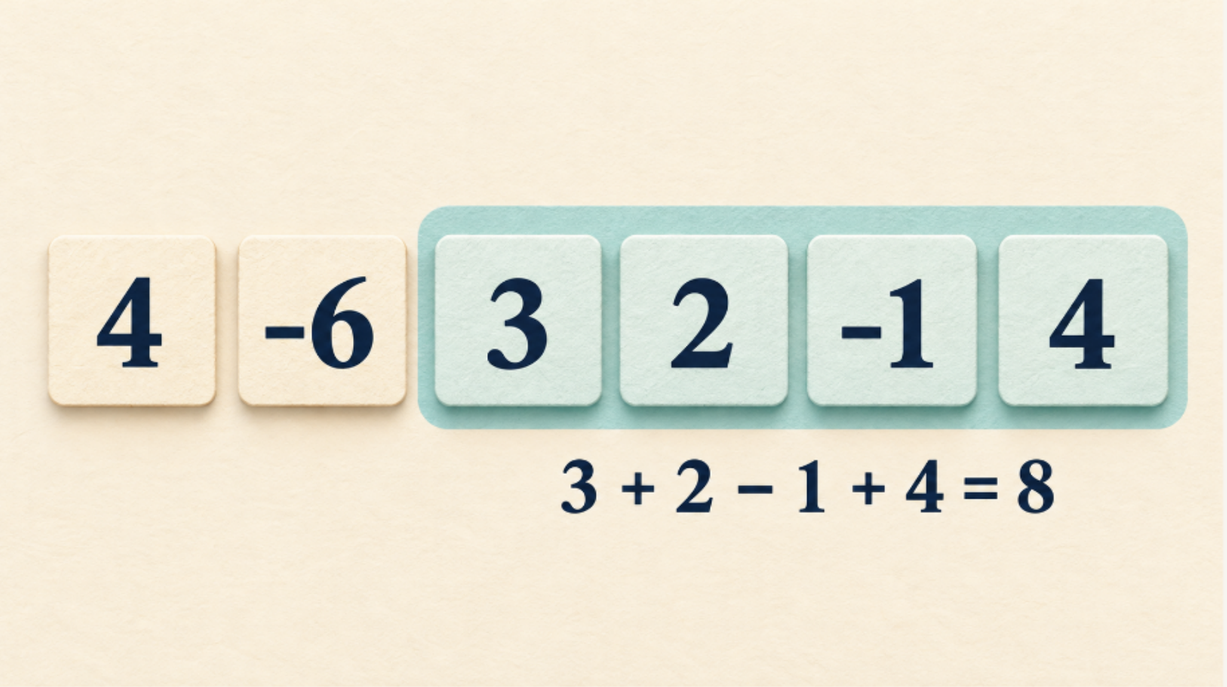
계단 계산을 직접 설명해 보세요

질문	먼저 떠올릴 내용
7칸의 답은 어느 두 답을 더할까?	6칸의 13가지 + 5칸의 8가지
0칸의 답은?	이동하지 않는 순서 한 가지
return dp[n]은 무엇을 돌려줄까?	요청한 높이 n의 방법 수

7칸의 답은 21가지입니다.

6-3 연속 부분합

이웃한 숫자를 한 덩어리로 골라, 합을 가장 크게 만듭니다.



입력: 정수 배열 a **구할 것: 연속한 구간의 최대 합**

중간 숫자는 건너뛸 수 없습니다

고른 숫자	허용 여부	이유
3, 2, −1, 4	가능	서로 이웃한 한 구간
3, 2, 4	불가능	중간의 −1을 건너뜀
4 하나	가능	길이 1인 구간
아무것도 고르지 않음	불가능	최소 하나를 골라야 함

선택한 시작점부터 끝점까지, 중간 숫자를 모두 포함합니다.

세 숫자라면 모든 구간을 써볼 수 있습니다

구간	합	3에서 끝나는가?
[4]	4	아니요
[−6]	−6	아니요
[3]	3	예
[4, −6]	−2	아니요
[−6, 3]	−3	예
[4, −6, 3]	1	예

전체 최고 합은 4. 마지막 3에서 끝나는 최고 합은 3.

첫 숫자 4부터 시작합니다

읽은 입력	4	−6	3	2	−1	4	이번 4
지금 끝나는 구간	4	−6	3	2	−1	4	합 4
지금까지 최고 구간	4	−6	3	2	−1	4	합 4

첫 숫자로 시작: 지금 끝나는 합 = 4, 지금까지 최고 합 = 4

-6을 읽으면, 지금 합과 최고 합이 갈라집니다

읽은 입력	4	**-6**	3	2	-1	4	이번 -6
지금 끝나는 구간	4	-6	3	2	-1	4	합 -2
지금까지 최고 구간	4	-6	3	2	-1	4	합 4

이어 붙이면 -2 **새로 시작하면 -6** **선택: -2**

이어 붙임: 4 + (-6) = -2. 전체 최고 4는 유지.

3은 앞에 붙이지 않고 새로 시작합니다

읽은 입력	4	−6	3	2	−1	4	이번 3
지금 끝나는 구간	4	−6	3	2	−1	4	합 3
지금까지 최고 구간	4	−6	3	2	−1	4	합 4

이어 붙이면 1 **새로 시작하면 3** **선택: 3**

이어 붙이면 1, 새로 시작하면 3. 더 큰 3을 선택.

2를 읽으면 앞 구간에 붙이는 편이 큽니다

읽은 입력	4	−6	3	**2**	−1	4	이번 2
지금 끝나는 구간	4	−6	3	2	−1	4	합 5
지금까지 최고 구간	4	−6	3	2	−1	4	합 5

이어 붙이면 5 **새로 시작하면 2** **선택: 5**

이어 붙이면 3 + 2 = 5. 지금까지 최고 합도 5.

-1을 읽어도, 이전 최고 합 5는 남깁니다

읽은 입력	4	−6	3	2	**−1**	4	이번 −1
지금 끝나는 구간	4	−6	3	2	−1	4	합 4
지금까지 최고 구간	4	−6	3	2	−1	4	합 5

이어 붙이면 4 **새로 시작하면 -1** **선택: 4**

지금 끝나는 합은 4. 지금까지 최고 합은 5.

마지막 4를 붙이면 답이 8로 바뀝니다

읽은 입력	4	−6	3	2	−1	**4**	이번 4
지금 끝나는 구간	4	−6	3	2	−1	4	합 8
지금까지 최고 구간	4	−6	3	2	−1	4	합 8

이어 붙이면 8 **새로 시작하면 4** **선택: 8**

3 + 2 − 1 + 4 = 8. 최종 반환값은 8.

지금 숫자에서 끝나는 구간은 두 방식으로 만듭니다

선택	이번 숫자가 3일 때	합
이번 숫자로 새로 시작	[3]	3
앞 구간에 이어 붙임	[4, -6] 다음에 3	-2 + 3 = 1

지금 끝나는 최고 합 = max(이번 숫자, 직전 끝나는 최고 합 + 이번 숫자)

두 합에 ending과 best라는 이름을 붙입니다

변수	뜻	-1을 읽은 직후
ending	지금 숫자에서 끝나는 최대 합	[3, 2, -1]의 합 4
best	지금까지 나온 최대 합	[3, 2]의 합 5

ending을 먼저 구하고, 이전 best와 비교해 best를 갱신합니다.

여섯 숫자를 읽은 기록

이번 숫자	앞 구간에 붙이면	새로 시작	ending	best
4	—	4	4	4
−6	4 − 6 = −2	−6	−2	4
3	−2 + 3 = 1	3	3	4
2	3 + 2 = 5	2	5	5
−1	5 − 1 = 4	−1	4	5
4	4 + 4 = 8	4	8	8

이번 ending = max(새로 시작, 이어 붙임)　　**이번 best = max(이전 best, 이번 ending)**

먼저 표 전체를 저장하는 코드로 옮기면

```
def max_subarray(a):
    ending = [0] * len(a)
    best = [0] * len(a)
    ending[0] = best[0] = a[0]
    for i in range(1, len(a)):
        ending[i] = max(a[i], ending[i-1] + a[i])
        best[i] = max(best[i-1], ending[i])
    return best[-1]
```

방금 본 표의 각 행을 ending[i], best[i]에 저장합니다.

다음 행에서는 바로 앞의 두 값만 읽습니다

단계	사용하는 값 / 계산	결과
시작 상태	직전 ending=4, best=5	이번 숫자=4
새 ending	max(4, 4 + 4)	8
새 best	max(5, 8)	8

표 전체가 아니라 바로 앞 행의 ending과 best만 기억해도 됩니다.

연속 부분합 Python 전체 코드

```python
def max_subarray(a):
    ending = a[0]
    best = a[0]
    for i in range(1, len(a)):
        ending = max(a[i], ending + a[i])
        best = max(best, ending)
    return best
```

초록색 ending을 먼저 갱신한 다음, 주황색 best를 갱신합니다.

best는 새 ending과 비교해야 합니다

차례	계산	결과
시작 상태	ending=3, best=4, 새 숫자=2	
1. ending부터	max(2, 3+2)	ending=5
2. best를 갱신	max(4, 새 ending 5)	best=5

계산 순서: 새 ending을 구한 뒤 새 best를 구합니다.

C는 숫자 배열과 길이를 함께 받습니다

C 표현	뜻	예시
const int a[]	읽기만 할 정수 배열	4, −6, 3, 2, −1, 4
int n	원소 개수	6
a[0]	첫 원소	4
a[n-1]	마지막 원소	a[5] = 4

long long max_subarray(const int a[], int n)

연속 부분합 C 전체 코드

```
long long max_subarray(const int a[], int n) {
    long long ending=a[0], best=a[0];
    for (int i=1; i<n; ++i) {
        long long extend=ending+a[i];
        if (extend>a[i]) ending=extend;
        else ending=a[i];
        if (ending>best) best=ending;
    }
    return best;
}
```

첫 if: 지금 끝나는 구간 선택　　**둘째 if: 전체 최고 합 갱신**

합을 담는 변수도 long long입니다

저장할 값	최대 예시	자료형
입력 원소 하나	1,000,000	int
원소 10,000개의 합	10,000,000,000	long long
번호 i와 개수 n	최대 10,000	int

숫자 하나의 크기와, 여러 숫자를 더한 합의 크기를 구분합니다.

모두 음수여도 하나는 골라야 합니다

읽은 숫자	ending	best
−8	−8	−8
−3	max(−3, −11) = −3	−3
−5	max(−5, −8) = −5	−3

초기값은 0이 아니라 첫 원소 a[0]입니다.

짧은 입력으로 코드의 뜻을 확인합니다

입력	선택한 구간	반환값
[-7]	[-7]	-7
[2, 4, 1]	[2, 4, 1]	7
[-2, 0, -1]	[0]	0
[4, -6, 3]	[4]	4

숫자 하나, 모두 음수, 0, 마지막에 최고가 나오지 않는 경우를 확인합니다.

두 문제에서 공통으로 한 일

문제	기억한 답	다음 계산에 사용
계단 오르기	앞의 두 높이의 방법 수	두 수를 더함
연속 부분합	직전 ending과 best	새 ending, 새 best를 구함

동적 계획법 DP: 작은 단계의 답을 저장하고 다음 답을 계산

전체 최고 합 하나만 기억하면 부족합니다

4, -6, 3까지 읽은 상태	값	다음 2에 바로 붙일 수 있나?
전체 최고 best: [4]	4	중간 숫자를 건너뛰므로 안 됨
현재 끝나는 ending: [3]	3	가능. [3, 2]의 합은 5

다음 계산에 필요한 답이 무엇인지부터 정확히 정합니다.

이제 두 함수를 직접 작성해 보세요

사이트 문제	먼저 손으로 확인	구현할 함수
6-2 계단 오르기	n=4일 때 5가지 바텀업(반복문)으로 작성	climb_stairs
6-3 연속 부분합	[4, –6, 3, 2, –1, 4]에서 8	max_subarray

https://sangdon-park.github.io/algorithm-lab/?chapter=6